

Dívida de processo e estagnação na assimilação em uma linha de produtos de software: um estudo longitudinal de pesquisa-ação

Rhuan Queiroz
PPCA, Universidade de Brasília
Brasília, Brasil
rhuancarlos.queiroz@gmail.com

André Barros de Sales
PPCA, Universidade de Brasília
Brasília, Brasil
andrebdes@unb.br

Resumo

Plataformas de software baseadas em *microfrontends* podem evoluir, de maneira *bottom-up*, para linhas de produtos de software (LPS) extrativas, cuja evolução, junto ao crescimento da utilização, levanta desafios de gestão de funcionalidades, como o inventário, o ciclo de vida, as interações e a adoção. Embora existam diversos estudos sobre a extração e a evolução de LPS, as intervenções que ocorrem durante essa evolução, o que é priorizado e o que é sistematicamente deixado para trás, não estão caracterizados. A partir de três ciclos articulados de pesquisa-ação em uma plataforma de *microfrontends* que opera como LPS extrativa de baixa maturidade, este trabalho contribui com: (C1) dívida de processo em LPS, proposta como mecanismo causal candidato e caracterizada longitudinalmente pela despriorização sistemática de práticas de gestão de funcionalidades, mesmo quando há capacidade conceitual e instrumento implementado; e (C2) estagnação multi-estágio na assimilação em LPS extrativa, padrão descritivo em que intervenções de melhoria distintas param em estágios distintos do modelo de assimilação, com os artefatos não passando da Adaptação e da Aceitação enquanto a mudança de processo fica retida na Adoção, heterogeneidade observável somente por meio de pesquisa-ação longitudinal. Por derivarem de um único caso longitudinal, os dois são apresentados como construtos candidatos e refutáveis, a serem testados por replicação em outras LPS extrativas.

Palavras-chave

Linhas de Produtos de Software, Gestão de Variabilidade, Gestão de Funcionalidades, Dívida de Processo, Pesquisa-Ação Longitudinal

1 Introdução

Neste trabalho, será abordada uma plataforma de software específica, que por definição é composta por um núcleo de funcionalidades, um conjunto de interfaces e um ecossistema de desenvolvedores e usuários. Enquanto que algumas definições enfatizam a criação de produtos a partir de partes reutilizáveis [29] e outras o conjunto de interfaces para a construção de outros sistemas sobre ela [33].

Essa, baseada em *microfrontends*, que evoluiu para uma linha de produtos de software (*Software Product Lines*). Marques et al. [17] abordam SPL à reutilização de código e artefatos de software, como requisitos, arquitetura e testes. A plataforma em questão, não partiu de uma abordagem *top-down*, mas sim de uma abordagem *bottom-up*, onde a plataforma evoluiu a partir da viabilização pela arquitetura de *microfrontends*. Sendo uma arquitetura capaz de combinar diferentes tecnologias, partes do *frontend* de maneira distribuída, para que seja possível a construção de software.

Dada a forma como se organizou a plataforma, ela se encaixa na definição de LPS extrativa, em que também há a vinculação em

tempo de execução e baseada em parâmetros, conforme a classificação de Apel et al. [1]. A evolução da plataforma, focando nos objetivos do negócio, junto ao crescimento da utilização, levou a uma série de desafios relacionados à gestão de funcionalidades, como o inventário de funcionalidades, ciclo de vida das funcionalidades, e interações entre as mesmas. Mas que também levam à discussão sobre a adoção das funcionalidades, ou seja, a adesão dos times de aplicação a utilizarem as funcionalidades disponíveis.

Existem diversos estudos sobre a extração e evolução de linhas de produtos de software, mas as intervenções que ocorrem durante a evolução, o que é priorizado e o que é sistematicamente deixado para trás, não estão caracterizados. Três lacunas são identificadas, todas em LPS Extrativa: (a) há diversos estudos empíricos em LPS [8], sendo apenas um dos trabalhos abordando LPS Extrativa (b) dívida de processo durante a evolução (literatura sobre dívida técnica); (c) abordar os estágios de assimilação por Cooper & Zmud [10] frente a mudanças [36] produzidas por intervenções de melhoria.

A partir de três ciclos articulados de pesquisa-ação em uma plataforma de *microfrontends* operando como LPS extrativa, este trabalho contribui com: (C1) Dívida de processo em LPS, proposta como mecanismo explicativo candidato e caracterizada longitudinalmente pela despriorização sistemática de práticas de gestão de funcionalidades, mesmo quando há capacidade conceitual e instrumento implementado. (C2) Estagnação multi-estágio na assimilação em LPS extrativa, padrão descritivo em que intervenções de melhoria distintas param em estágios distintos do modelo de assimilação, heterogeneidade observável somente via pesquisa-ação longitudinal. Por derivarem de um único caso, os dois são apresentados como construtos candidatos e refutáveis: replicações em outras LPS extrativas podem confirmá-los, refiná-los ou refutá-los.

2 Fundamentação

2.1 Linhas de Produto de Software Baseadas em Parâmetros

Uma LPS pode surgir de maneira *bottom-up*, a partir de plataformas ad-hoc, por meio de um processo de extração que comumente requer a análise dos artefatos existentes [19], para o qual há *frameworks* e abordagens estruturadas [19, 20]. Dentre os diferentes tipos de artefatos compartilhados, o mais comum sendo código, também é possível observar requisitos, componentes, documentação e modelos [17, 18]. Uma variante, por sua vez, é uma combinação válida de funcionalidades [18].

Um dos problemas pertinentes em extrações relaciona-se ao modelo de funcionalidades, cujo processo requer a evidencição dessas funcionalidades, seja por identificação ou localização [19, 21], bem como das dependências, restrições e interações entre elas [27],

sendo um grande desafio para o processo de extração de LPS. As interações entre funcionalidades, por definição, referem-se a quando o comportamento de uma funcionalidade é afetado por outra [2].

2.2 Dívida de Processo

Dentre as diferentes formas de dívida abordadas na literatura, a dívida técnica é a mais conhecida e estudada, e se refere a decisões técnicas que priorizam a entrega rápida em detrimento da qualidade do código, também sendo abordada em alguns estudos junto com outras dívidas, como a de processo [23], que é definida de maneira análoga como decisões subótimas em nível de processo, focadas no longo prazo, mas que podem levar a consequências negativas no longo prazo. O que caracteriza e diferencia as dívidas de processo das dívidas técnicas (que vivem no código) é que a suboptimalidade está na prática (entradas, papéis, handoffs, dependência de domínio), e não no código.

2.3 Assimilação de Inovações: O Modelo de Cooper e Zmud

O modelo de Cooper & Zmud [10] descreve a assimilação de tecnologias e inovações em seis estágios:

- (1) **Iniciação (Initiation)**: a equipe reconhece uma necessidade e identifica que uma nova tecnologia, instrumento ou prática pode atendê-la;
- (2) **Adoção (Adoption)**: negocia-se o respaldo organizacional por meio de deliberação e avaliação entre os envolvidos, sendo o produto do estágio a decisão de investir e comprometer recursos com a implementação, dando aval para avançar;
- (3) **Adaptação (Adaptation)**: organização e equipe passam por ajustes mútuos para encaixar o instrumento nos fluxos de trabalho existentes;
- (4) **Aceitação (Acceptance)**: a equipe revê suas práticas para incorporar a inovação, superando a inércia do paradigma anterior;
- (5) **Rotinização (Routinization)**: o uso deixa de ser novidade e se incorpora ao trabalho cotidiano, substituindo as práticas anteriores;
- (6) **Infusão (Infusion)**: o estágio mais avançado, em que a inovação é explorada além do previsto.

Uma distinção do modelo é especialmente relevante para a análise deste trabalho: na Adaptação o instrumento está construído e disponibilizado para uso, ao passo que na Aceitação ele passa a ser de fato empregado no trabalho. Disponibilizar e empregar são dois produtos distintos, e a passagem de um para o outro não é automática.

3 Trabalhos Relacionados

3.1 LPS Extrativa

Abordagens de implementação de LPS são amplamente discutidas na literatura, em especial as que surgem de maneira *bottom-up* a partir de plataformas ad-hoc, como é o caso do presente trabalho e não diferente de outros estudos [15, 19, 20] e correspondendo a cerca de 60% dos estudos nos últimos 20 anos[3]. Alguns autores observam que o processo de extração na indústria é o momento mais comum para a inserção de débitos técnicos [37].

Um dos indutores para a criação de uma LPS comumente é a necessidade de reutilização, ou até a própria variabilidade, onde a extração de uma LPS é uma maneira de organizar e gerenciar a variabilidade para necessidades do mercado [6]. Mas também outros indutores podendo estar relacionados a diferentes tipos de sustentabilidade [9], algo muito próximo de uma necessidade vinda de otimização de recursos, onde projetos podem ser mantidos de formas mais eficientes[13], uma forma de reutilização de código.

3.2 Dívida de Processo

A dívida de processo, definida na Seção 2.2, embora seja um conceito conhecido, é recém-caracterizada na literatura.

Dito isso, o presente trabalho também relata como as dívidas de processo se manifestam em uma LPS extrativa de baixa maturidade, onde a pressão de priorização organizacional produz a despriorização sistêmica de práticas de gestão, o que pode levar a consequências negativas no longo prazo ou mesmo a um ciclo vicioso de mais dívidas. Sendo a despriorização organizacional corroborada por Martini et al. [22] como uma das causas de dívidas de processo, junto com a negligência do valor, falta de acompanhamento e também competência com processos por parte dos praticantes. Até onde alcança nossa revisão, a dívida de processo não foi caracterizada especificamente no contexto de LPS extrativa.

Alguns estudos de caso abordam outros tipos de dívidas, como a dívida técnica no contexto de linhas de produtos de software da indústria [39, 40], destacando as principais causas de dívida técnica, como a complexidade da base de código, falta de tempo para desenvolvimento e decisões de negócio [39]. E também argumentando como as evoluções de processo podem levar a dívidas técnicas[40], o que pode ser um caminho para trabalhos futuros, considerando a relação entre as dívidas de processo e técnica, e como elas se manifestam em LPS extrativas. Algo que está relacionado ao direcionamento de recursos para o processo, levando à falta de recursos para o desenvolvimento, que conversa muito bem com os achados de Green & Hevner [14] sobre como a adoção de uma inovação (aqui no caso a mudança no processo) pode ser um grande distrativo para o trabalho. De modo semelhante, ocorreram mudanças ao longo dos ciclos deste trabalho, onde a inovação (aqui a evolução do modelo de funcionalidades) pode ter levado à perpetuação do débito de processo identificado.

3.3 Adoção e Assimilação de Inovações

Green & Hevner [14] abordam como a métrica chave para o sucesso da difusão de uma inovação é o seu uso. Também abordam como muitas das decisões de adotar ou não uma inovação são decididas no nível gerencial, onde a adoção passa a ser obrigatória para as camadas abaixo. O processo de adoção é complexo, envolve vários fatores mediadores.

Essa dimensão organizacional e frequentemente mandatória da adoção reaparece em trabalhos posteriores. Numa perspectiva mais futurista, um estudo recente [25] aborda as implicações relacionadas à adoção de ferramentas baseadas em inteligência artificial, conduzida com 38 estudantes e utilizando cenários narrativos futuristas, também aborda o tópico de como essas adoções comumente são mandatórias por parte da organização[14], e como isso pode impactar a adesão à inovação adotada, dimensão que este trabalho

observa sob a ótica dos estágios de assimilação, embora aqui não se trate de ferramentas de IA, mas de um cenário em ascensão.

Os principais achados de Melegati et al. [25] foram: (1) desenvolvedores podem se sentir substituídos por ferramentas de IA e (2) ferramentas baseadas em IA podem influenciar a autopercepção dos desenvolvedores. Também traz uma generalização de como a adoção de ferramentas, não só de IA, deve considerar os potenciais desafios psicológicos ao implementar ferramentas que mudam drasticamente a forma de trabalhar. E como esses desafios psicológicos mudam entre diferentes profissionais.

Um segundo conjunto de estudos chama atenção para a dependência do contexto na adoção de ferramentas. Um estudo sobre pequenas companhias de software [35], por um mapeamento sistemático seguido pela construção de um framework de adoção, traz como a adoção de ferramentas é ainda mais difícil em pequenas companhias, onde argumentam que essas ferramentas só são úteis quando o contexto em que as companhias estão inseridas é levado em conta. O presente trabalho se contextualiza numa grande companhia, mas que tem uma série de times pequenos e independentes, o que pode levantar a discussão sobre como os achados podem ser aplicados em times menores.

Já nos contextos de variabilidade, dois trabalhos convergem no problema da adoção da gestão de variabilidade em linhas de produtos de software na indústria/prática, diferindo sobretudo no tipo de artefato examinado: ferramentas [4], num caso, e técnicas [6], no outro. A revisão da adoção de ferramentas de gestão de variabilidade [4, 5] traz que muitas ferramentas são adotadas na prática, mas que funcionam melhor no contexto em que foram criadas, sendo difícil a adoção em outros contextos. Argumentam que o baixo uso de ferramentas vindas da academia pode ser devido a uma presença de uma grande variedade de ferramentas “caseiras” na indústria, que são desconhecidas pelos pesquisadores.

De modo muito semelhante, o trabalho sobre a adoção de técnicas de gestão de variabilidade [6] considerou 12 casos (média-grande escala) na indústria, um dos desafios mencionados é a integração de diferentes ferramentas às ferramentas de engenharia dos contextos específicos, argumentando que a adoção é um problema de integração de ferramentas. Esse achado pode, inclusive, justificar a criação das próprias ferramentas[4] para evitar e contornar os desafios de integração, que também conversa com a preocupação da consideração de contextos específicos[35].

Um outro desafio abordado por Berger et al. [6], refere-se ao processo de migração, onde este deve ser leve, guiando os engenheiros durante a migração, sem interromper o desenvolvimento. Que conversa bem com um achado de [14] argumentando como as mudanças ao modificarem o processo de trabalho enquanto necessitam que o trabalho continue sendo feito, podem ser um grande distrativo para a migração.

O presente trabalho, embora não seja focado nos detalhes das ferramentas em si, traz evidências que corroboram a criação de ferramentas “caseiras” antes de adotar ferramentas externas, mas sem abordar as motivações por trás disso, o que pode ser um caminho para trabalhos futuros, e traz também evidências pré-adoção de como a preocupação com a migração e a integração já aparece antes mesmo da adoção.

4 Metodologia

Neste trabalho, conduzimos uma pesquisa-ação embutida em um estudo de caso longitudinal único, adotando as recomendações de Petersen et al. [28] para a condução de pesquisa-ação em parceria indústria-academia. Utilizando também o vocabulário canônico de cinco fases de Susman & Evered [32], ao qual as fases de Tripp [34], que orientaram o desenho dos ciclos, são mapeáveis; e as diretrizes de Runeson & Höst [31] para estudos de caso em engenharia de software.

Entende-se a pesquisa-ação (PA) como uma metodologia de investigação que combina ação prática e reflexão teórica, algo que é abordado até como duas metodologias que fazem parte do trabalho final[34]. PA fornece um ciclo iterativo de intervenções e geração de conhecimento, enquanto o estudo de caso oferece uma estrutura para a análise detalhada de um fenômeno específico dentro de seu contexto real.

Para fins de legitimação metodológica, levanta-se a discussão sobre o duplo-ciclo de pesquisa e problema[24], onde se reporta o ciclo de pesquisa e não o ciclo do problema, onde os critérios operacionais são discutidos [34] focados no alvo principal da pesquisa, que é a geração de conhecimento, e não a resolução do problema. Onde em cada ciclo há a reflexão sobre: (a) o que foi aprendido sobre PA; (b) o que foi aprendido sobre a resolução do problema[24]. Os ciclos partem de uma questão prática que envolve todos os participantes, e é a partir do foco da equipe em resolver o problema que os efeitos da intervenção são analisados e refletidos[34].

Uma pesquisa inicial sobre PA em Engenharia de Software[11] aponta o foco de trabalhos em melhoria de processos e a importância de tornar a PA recuperável, o análogo de replicável, por meio da declaração antecipada da metodologia. Se executada corretamente, PA pode levar à situação benéfica de geração de conhecimento e resolução de problema[12], também muito útil para investigar fenômenos não homogêneos em relação ao tempo.

4.1 Estudo de Caso

Seguindo as recomendações de Runeson & Höst [31] e a tipologia de Yin [38], o propósito do estudo de caso é a melhoria do fenômeno estudado, mas ao mesmo tempo, também é descritivo por relatar um fenômeno específico, situado no quadro teórico de LPS extrativa baseada em parâmetros [1] e detalhado na Seção 5.

Acerca da unidade de análise, é o time de domínio, que é o time da plataforma, que controla a governança da plataforma e também da linha de produtos, as subunidades de análise são os três ciclos de pesquisa-ação, que são lidos transversalmente, e times adjacentes, que também estão envolvidos na plataforma. Distinta da unidade de análise[38], a unidade de observação da leitura transversal é a mudança ($n = 5$): a estagnação é medida sobre a mudança, não sobre o caso ou suas subunidades.

A seleção do caso atende a duas das justificativas canônicas de Yin [38]: **caso revelador**, pouco acesso à intersecção de linhas de produto, gestão de variabilidade, *microfrontends* e pesquisa-ação. E **caso longitudinal**, o acompanhamento do fenômeno de gestão da linha de produtos bem como sua evolução ao longo do tempo.

Parte do objetivo desse estudo é a generalização analítica[38] dos fenômenos para a teoria, extensão e refinamento de construtos sobre

evolução de LPS extrativa. Não se busca a generalização estatística para toda a população de LPS.

4.2 Ciclos de Pesquisa-Ação

Os ciclos de pesquisa-ação seguem o modelo de cinco fases[32], a Figura 1 apresenta a estrutura geral dos ciclos, a pergunta-chave e as mudanças propostas. Operacionalmente, as fases de Tripp distribuem-se em dois campos, prática e investigação da prática. A avaliação da prática não foi considerada, aproximando ainda mais a prática da prática canônica, então, foi possível mapear as fases de Tripp[34] para as fases de Susman & Evered[32]. Cada ciclo é relatado em quatro movimentos: diagnóstico; intervenção (planejamento e ação); avaliação; e desfecho e aprendizagem, com a aprendizagem em duas camadas, a prática, que encadeia os ciclos, e a transversal, de pesquisa.

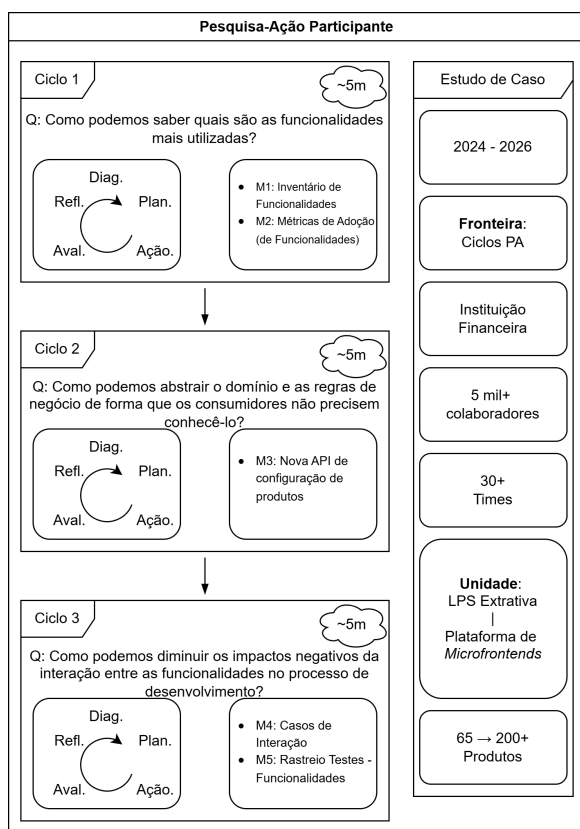


Figura 1: Resumo da metodologia do trabalho; o rótulo ~5m indica a duração aproximada de cada ciclo, em meses.

Além disso, é interessante para organizar a melhoria da prática, pois já é um processo que ocorre normalmente[28].

4.3 Coleta e Análise de Dados

Ao longo dos ciclos foram utilizados diferentes tipos de instrumentos de coleta, como: análise estática de artefatos, questionários, grupos focais e observação participante prolongada (útil para pegar dados em interações informais [28]). Os instrumentos aplicados em

cada ciclo, com seus respectivos tamanhos amostrais, estão sintetizados na Tabela 1. A observação participante estendeu-se para além do encerramento formal dos ciclos: o pesquisador permaneceu vinculado à plataforma, o que permitiu aferir o estado de cada intervenção em uma janela posterior e distinguir intervenções ainda em curso daquelas cujo desfecho se consolidou, situação relatada ao final da Seção 6. A análise das transcrições dos grupos focais seguiu a abordagem de análise temática de Braun & Clarke [7], com triangulação intra- e inter-ciclo. Os códigos resultantes seguem um esquema que identifica a questão do roteiro, o tema e o código dentro do tema (por exemplo, Q2-T4-C1), e é esse identificador que acompanha os trechos citados na Seção 7, de modo que cada fala possa ser localizada no *codebook* depositado.

5 Contexto

O contexto deste estudo é uma instituição financeira de grande porte, que emprega milhares de funcionários no Brasil. A organização adota uma estrutura de times multidisciplinares, auto-organizados e independentes entre si, que acabam por clamar o baixo acoplamento entre eles. A adoção de uma arquitetura de *microfrontends* visava atingir ainda mais autonomia e independência entre os times, ao mesmo tempo em que promovia o reuso e a composição de aplicações web. Sendo assim, constitui-se a plataforma de *microfrontends*, composta por um conjunto de ferramentas, componentes e recursos disponíveis para os times, reutilizarem e criarem suas aplicações sobre a plataforma; envolvendo centenas de aplicações, que são os produtos da linha, e dezenas de times.

Sobre a arquitetura da plataforma, ela é composta por um núcleo (*core*) de componentes, sendo eles a *Shell*, que é responsável por orquestrar a renderização e composição dos *microfrontends* em tempo de execução, e a API de Regras, que expõe a folha de regras (mecanismo de variabilidade) para as aplicações. Além disso, existem *microfrontends* comuns, como login, header, sidebar e footer, que são reutilizados por diversas aplicações. Já as aplicações, por sua vez, são também *microfrontends* específicos, que são desenvolvidos de forma independente e descentralizada pelos times de aplicação, mas ainda assim, são renderizados pelo *Shell* da plataforma.

O mecanismo de variabilidade da plataforma é a folha de regras, um artefato JSON exposto pela API de Regras, que define quais componentes serão renderizados, e quais funcionalidades e sub-funcionalidades estarão habilitadas para cada aplicação. A folha de regras é um artefato flexível, sem rigidez de esquema, o que significa que não há um padrão pré-definido para a adição de novas funcionalidades. Essa flexibilidade, embora permita a evolução da plataforma, também traz desafios para a gestão e configuração das funcionalidades, que é o principal problema do segundo ciclo de PA deste estudo. Cabe distinguir os termos que nomeiam as camadas desse mecanismo, haja vista que reaparecem ao longo do texto: o *gerenciador de regras* é a ferramenta interna em que as regras são mantidas; a *folha de regras*, também referida como *arquivo de configuração* quando tomada do ponto de vista de um produto, é o artefato JSON resultante; e a *API de Regras* é o serviço que a expõe às aplicações em tempo de execução.

Embora a organização não utilize formalmente o termo *LPS*, a arquitetura da plataforma exhibe características centrais de uma

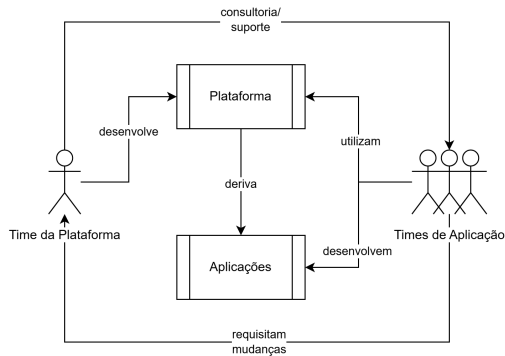


Figura 2: Resumo do fluxo de trabalho entre os times da plataforma e aplicações.

LPS extrativa, reativa/incremental[26] (embora as duas sejam diferentes, a primeira explica a origem da LPS e a segunda, o modo de evolução), com binding em tempo de execução, mecanismo baseado em linguagem (baseado em parâmetros) e derivação composicional, conforme a taxonomia de Apel et al. [1].

Os papéis envolvidos na plataforma assemelham-se à engenharia de domínio (time da plataforma) e à engenharia de aplicação (times consumidores), embora a organização não adote formalmente uma dicotomia direta entre esses papéis, e haja questões de semelhança e sobreposição entre eles. A Figura 2 apresenta um resumo do fluxo de trabalho entre os times.

O time da plataforma também presta consultoria e suporte aos times de aplicação, por meio de bilhetes de atendimento, que por muito tempo foram o principal canal para a configuração das aplicações, mas que atualmente são utilizados em casos específicos ou para configurações complexas. Sendo as configurações mais simples realizadas pelos próprios times de aplicação por meio de uma aplicação de autosserviço, que é desenvolvida por outra equipe, paralela ao time da plataforma.

Do ponto de vista prático da organização, são duas lacunas: a ausência de um modelo de funcionalidades formal, onde as funcionalidades não são documentadas, nem os detalhes de seus relacionamentos são explicitados. E a dificuldade com a baixa adoção das funcionalidades, que representa um desafio quanto aos recursos utilizados e consequentemente uma barreira para a evolução proativa da plataforma.

6 Trajetória dos Ciclos em Síntese

Os ciclos de PA-EC foram conduzidos em sequência e, tomados em conjunto, registram a trajetória do paradigma de gestão de funcionalidades na organização, onde o estado em que cada ciclo termina é o ponto de partida do seguinte. No início, a gestão era puramente ad-hoc, baseada no conhecimento tácito dos indivíduos acerca das funcionalidades e sem um inventário formal. O primeiro ciclo traz uma intervenção para criar um inventário estruturado e identificar as funcionalidades presentes na base de código. Já para o segundo ciclo, a organização buscou abstrair o domínio e as regras de negócio, automatizando a geração dos arquivos de configuração a partir de um esquema de validação, dados e baseado em *templates*.

O terceiro ciclo, por sua vez, tem como foco as interações entre funcionalidades, um desafio que se tornou mais evidente à medida que o produto apresentava falhas que passavam despercebidas.

As intervenções de cada ciclo são apresentadas em síntese, seu tratamento como contribuições autônomas é objeto de trabalhos separados, um deles já aceito para publicação[30] e os demais em elaboração ou revisão. Aqui, os eventos dos ciclos sustentam os achados transversais discutidos na Seção 7. A Tabela 1 sintetiza, para cada ciclo, a pergunta, o diagnóstico, a intervenção, os instrumentos de avaliação e o desfecho.

6.1 Ciclo 1: Inventário de Funcionalidades e Métrica de Adoção

Diagnóstico. O conhecimento sobre as funcionalidades era tácito, pertencente aos indivíduos e não à organização como um todo. Ao mesmo tempo, que os integrantes dos times de aplicação também não tinham esse conhecimento, o que ficava restrito à passagem de conhecimento por meio dos bilhetes de atendimento e consultoria. Partindo então da hipótese assumida no ciclo, de que a ausência desse conhecimento explícito e estruturado poderia ser um fator limitante para a adoção das funcionalidades, a pergunta do ciclo foi: *como podemos saber quais são as funcionalidades mais utilizadas?*

Intervenção. Uma intervenção em quatro movimentos foi conduzida: (i) análise estática do código para identificar e localizar as funcionalidades; (ii) inventário estruturado, disponibilizado no gerenciador de regras, com atributos como o domínio, descrição, id único e caminho *JSONPath* no arquivo de configuração; (iii) execução automatizada semanal da análise dos arquivos de configuração; (iv) estatística descritiva de uso por cada arquivo de configuração.

Avaliação. O questionário contou com a participação de 10 integrantes, o que representa cerca de 50% do time, e o formulário ficou aberto por 27 dias. A amostra tem um erro amostral de 22% a 95% de confiança¹, que leva a uma interpretação descrita como “indicativa” dos resultados. A percepção de relevância da lista de funcionalidades foi alta, com uma média de 4,4 em uma escala de 1 a 5, enquanto o esforço necessário percebido para adotar a lista foi de 2,2. No entanto, houve um contraste expressivo entre a percepção de relevância e o uso declarado, com cerca de 33% dos participantes afirmando usar a lista, e mesmo entre aqueles que configuraram produtos, havia um desconhecimento da lista.

Desfecho. A lista de funcionalidades teve um uso pontual e fora de rotina: orientou a preservação do comportamento na migração de um componente de ReactJS para Angular. O instrumento chegou a ser implantado e integrado ao gerenciador de regras, mas foi posteriormente despriorizado frente a outros problemas envolvendo a gestão de funcionalidades. A gestão da plataforma, por sua vez, deixou de acompanhar as métricas de adoção.

Aprendizagem. O registro do ciclo deslocou o foco de “medir a adoção” para a gestão de funcionalidades como um todo. Esse deslocamento decorre do que o ciclo registrou: gestão e adoção, embora

¹Erro amostral para proporção com correção de população finita, $e = z\sqrt{p(1-p)/n} \cdot \sqrt{(N-n)/(N-1)}$, com $z = 1,96$ (95% de confiança), $p = 0,5$ (caso mais conservador), $n = 10$ e população assumida $N \approx 20$, o tamanho do time à época.

Tabela 1: Visão geral dos três ciclos de pesquisa-ação.

Ciclo	Pergunta do ciclo	Diagnóstico	Intervenção	Instrumentos (n)	Desfecho
1	<i>Quais funcionalidades são as mais utilizadas?</i>	Conhecimento tácito e individual; sem inventário formal.	Análise estática do código; inventário estruturado no gerenciador de regras; métrica semanal de adoção.	Questionário (n=10; ≈50% do time; erro amostral 22%).	Implantado, depois despriorizado. Relevância percebida alta (4,4/5) ante desconhecimento da lista.
2	<i>Como abstrair o domínio e as regras para que os consumidores não precisem conhecê-los?</i>	Configurar exige conhecer o domínio; edição manual propensa a erros; acoplamento entre os times.	Nova API que gera o arquivo de configuração a partir de esquema de validação, dados e templates; prova de conceito com retrocompatibilidade.	Dois grupos focais (plataforma n=13; consumidor n=14).	Prova de conceito validada e adotada pela organização; migração das regras existentes como desafio futuro.
3	<i>Como reduzir os impactos negativos da interação entre funcionalidades no desenvolvimento?</i>	Interações inesperadas, estimadas em 1 a 3 por mês e toleradas; falhas percebidas apenas em produção.	(i) casos de uso problemáticos na ferramenta de experimentação; (ii) vínculo entre funcionalidade e suite de testes no processo de desenvolvimento.	Grupo focal (n=10) e observação participante.	Prova de conceito apenas da parte (i); mudança do processo de desenvolvimento não implementada.

próximas, podem se ofuscar mutuamente, e as métricas de adoção acabavam por encobrir os problemas internos de gestão. Soma-se a isso o reconhecimento, por parte da gestão da plataforma, de que configurar as funcionalidades ainda exigia conhecer o domínio e era feito de forma manual, preocupação que alimenta diretamente o diagnóstico do ciclo seguinte. Do ponto de vista metodológico, o formulário não se mostrou um instrumento eficaz de coleta, o que levou a considerar outros instrumentos para a avaliação das intervenções futuras.

6.2 Ciclo 2: Automação da Configuração de Produtos

Diagnóstico. A questão de configurar os produtos exige um conhecimento prévio do domínio, o que torna a edição manual propensa a erros, ao mesmo tempo que um dos times responsáveis por uma aplicação de autosserviço para configurar produtos, que consome a API de configuração, também carecia desse conhecimento para a manutenção da aplicação. Com isso, gerava-se um acoplamento entre esses dois times que atuam na construção da plataforma, que levou à questão: *como podemos abstrair o domínio e as regras de negócio de forma que os consumidores não precisem conhecê-los?*

Intervenção. Uma nova API foi proposta, que gera o arquivo de configuração a partir da evolução da estruturação das funcionalidades do ciclo 1, onde é possível compor regras a partir de um esquema de validação e por meio de template *engine* gerar partes canônicas do arquivo de configuração. A proposta foi implementada como uma prova de conceito, mantendo a restrição de retrocompatibilidade com as regras existentes.

Avaliação. A avaliação contou com dois grupos focais, um com 13 participantes da plataforma e duração de cerca de 40 minutos, e outro com 14 participantes consumidores da API, com duração de cerca de 30 minutos. Além disso, um formulário foi disponibilizado, mas teve baixa adesão, com apenas 7 respostas, o que levou ao seu descarte. Em síntese, os benefícios de abstração e desacoplamento foram reconhecidos pelos participantes, enquanto a dificuldade em lidar com a abstração e a preocupação com a migração foram registradas como desafios. Os detalhes temáticos (pré-adoção) são objeto de trabalho separado[30].

Desfecho. A proposta foi validada como prova de conceito e a organização decidiu adotá-la e evoluir as implementações dela

decorrentes, com um redirecionamento da capacidade e do foco da equipe para esse serviço. A migração das regras existentes para a nova API, porém, seguia, à época do ciclo 3, como desafio futuro, com a ferramenta ainda em desenvolvimento.

Aprendizagem. A avaliação recaiu sobre uma prova de conceito, de modo que o que se captou foi a percepção de potencial da proposta, e não o valor já realizado pelo uso, o que circunscreve o alcance do que o ciclo pôde observar. Do ponto de vista metodológico, o formulário voltou a se mostrar um instrumento ineficaz de coleta, como já ocorrera no ciclo anterior, o que consolidou o grupo focal para a avaliação das intervenções. O ciclo registrou ainda uma virada de foco para as funcionalidades no centro, não só da gestão, mas da própria tecnologia: uma API antes voltada a arquivos de configuração passou a se organizar em torno das funcionalidades, ainda que a abstração do modelo, mais do que a construção da API em si, tenha sido a parte mais difícil. Esse novo arranjo torna-se o contexto que condiciona o diagnóstico do ciclo seguinte.

6.3 Ciclo 3: Interações entre Funcionalidades e Rastreabilidade de Testes

Diagnóstico. Mesmo com a mudança de contexto focada nas funcionalidades, em todos os níveis da organização, havia um deles que trazia preocupação e também desconforto ao time da plataforma. Durante o processo de desenvolvimento, era comum que surgissem interações inesperadas entre as funcionalidades, muitas delas eram até resolvidas ou já sabidas, mas com certa frequência, estimada em torno de 1 a 3 problemas por mês, surgiam interações que passavam despercebidas e ocorriam só em produção, o que dificultava o processo de desenvolvimento. O time tolerava essa frequência, era feito o *rollback* rapidamente e o item voltava para o estado de desenvolvimento, mas o fato de não conseguir antecipar essas interações e de elas passarem despercebidas era um ponto de atenção da gestão. Com isso, a pergunta do ciclo foi: *como podemos diminuir os impactos negativos da interação entre as funcionalidades no processo de desenvolvimento?*

Intervenção. Uma proposta em duas partes foi elaborada: a primeira parte consistia em estender a ferramenta interna de experimentação e testes, que é amplamente utilizada pelos times de desenvolvimento, com uma lista de casos de uso problemáticos

conhecidos, ou seja, casos em que a interação entre funcionalidades já havia causado problemas no passado. A segunda parte da proposta envolvia uma mudança no processo de desenvolvimento, vinculando as funcionalidades a suítes de teste específicas a cada desenvolvimento ou alteração, bem como mapear futuras interações. A execução da proposta resultou em uma prova de conceito apenas da primeira parte, realizada pelo próprio pesquisador.

Avaliação. Foi utilizada a mesma ideia do ciclo anterior de um grupo focal para avaliar as propostas, com a participação de 10 integrantes, e duração de cerca de 11 minutos, realizada de forma remota. A duração menor que a dos grupos focais do ciclo anterior decorre de uma pauta mais estreita, haja vista que as propostas já eram conhecidas dos participantes, discutidas ao longo do diagnóstico do próprio ciclo, restando ao grupo focal a avaliação em si; ainda assim, trata-se de uma coleta de menor profundidade, registrada aqui por transparência. Além disso, a avaliação contou com a observação participante do pesquisador. Os benefícios e a importância pré-adoção foram reconhecidos pelos participantes, seis temas foram identificados, mas também não serão listados aqui, pois compõem um trabalho separado, ainda não publicado.

Desfecho. A gestão reconheceu a importância do problema, os praticantes reconheceram a importância pré-adoção, mas a equipe no momento do ciclo 3 estava focada nas implementações decorrentes da proposta do ciclo 2. A parte (ii) da proposta do ciclo 3 não foi implementada por “resultados do ciclo anterior e outras prioridades da organização”, de modo que a intervenção do ciclo foi executada apenas em parte. Os problemas de interação entre funcionalidades continuam a ocorrer e seguem sendo tratados de forma reativa. A ferramenta de experimentação já é amplamente usada como documentação comportamental e viva das funcionalidades.

Aprendizagem. O ciclo registrou a percepção da equipe de que, neste contexto, mudanças de ferramentas são mais facilmente adotadas do que mudanças do processo de desenvolvimento, o que é um aprendizado importante para a organização. A ferramenta de experimentação faz parte de todo o ciclo de desenvolvimento e, apoiada na própria arquitetura da LPS, permite combinar funcionalidades de forma flexível, interativa e em tempo real.

6.4 Situação das Intervenções após os Ciclos

Para além do encerramento formal dos três ciclos, a observação participante prosseguiu, o que permitiu acompanhar o destino de cada intervenção em uma janela posterior. Até a redação deste trabalho, o inventário de funcionalidades e a métrica de adoção do ciclo 1 seguiam implantados, mas sem acompanhamento da gestão e com a métrica não sendo utilizada, embora disponível. A nova API de configuração do ciclo 2 foi construída e disponibilizada em produção, porém a migração das regras existentes não foi realizada, de modo que a equipe segue operando no paradigma anterior. A prova de conceito do ciclo 3, por sua vez, permaneceu em homologação e não foi incorporada à rotina, enquanto a mudança no processo de desenvolvimento não chegou a ser empreendida.

Nenhuma das cinco intervenções, portanto, foi incorporada à rotina de trabalho da equipe na janela observada, e são esses desfechos que sustentam os dois achados transversais discutidos a seguir.

7 Achados Transversais e Longitudinais

Os eventos da seção anterior, lidos em isolamento, parecem contingências locais; lidos em sequência, sugerem dois padrões estruturais: a dívida de processo em LPS extrativa (C1) e a estagnação multi-estágio na assimilação de intervenções de melhoria (C2). Por derivarem de um único caso, são propostos como construtos candidatos e refutáveis. Ademais, só são visíveis por meio da pesquisa-ação longitudinal: não emergiriam dos trabalhos de cada ciclo tomados separadamente.

C1 é uma contribuição causal, que propõe um mecanismo explicativo candidato para a dívida de processo em LPS extrativa sob capacidade finita. C2 é uma contribuição descritiva, que documenta um padrão de estagnação tipada por estágio na assimilação de intervenções de melhoria. Os dois não tomam o mesmo objeto, e é essa diferença que os mantém separados ao longo da seção: a unidade de C1 é o processo subótimo que persiste, sendo dois os caracterizados aqui, e a pergunta que se responde é por que ele não é pago; já a unidade de C2 é a mudança, sendo cinco as rastreadas, e a pergunta é em que estágio cada uma para. Ou seja, a dívida é uma prática que segue em operação, enquanto a estagnação é a posição de uma mudança na trajetória de assimilação.

7.1 Dívida de Processo em LPS

Ao olhar para os ciclos, é possível perceber de início, no diagnóstico do ciclo 1, que um processo de configuração manual das folhas de regra, e/ou semi-automatizado por meio da aplicação de autosserviço para configuração (D1), caracteriza-se como uma dívida de processo, conforme a Seção 2.2. Onde o processo subótimo é o fluxo de trabalho de configuração, e o custo recorrente, além do esforço manual (orientado a bilhetes), são os erros e retrabalho decorrentes. Ademais, há indício de que esse custo recorrente também seja percebido pela própria equipe, e não apenas pela leitura do pesquisador, haja vista que na avaliação do ciclo 2 um desenvolvedor do time da plataforma o remete ao quadro de trabalho do dia a dia: “se a gente abrir o quadro de atendimento vai ver que metade dele são issues de relação a regra” (Q3-T1-C1, ciclo 2). Trata-se de percepção do participante, e não de medida, mas sugere a ordem de grandeza que se atribui ao retrabalho.

Uma outra dívida desse tipo é o tratamento reativo de interações (D2), onde mesmo que sejam geradas falhas no ambiente de produção, o processo de lidar com elas é puramente reativo, sem mapeamento ou prevenção, aceita-se pagar o custo recorrente de lidar com alguns desses incidentes por mês, o que caracteriza o segundo processo subótimo. Em ambos os casos, a suboptimalidade está na prática, e não no código, conforme distingue a Seção 2.2: a nova API do ciclo 2 muda diretamente a prática, e não se propõe a consertar um atalho do passado ou uma decisão arquitetural.

Ainda assim, D1 é perpetuada porque a melhoria que se mostra válida para pagar a dívida, a nova API de regras, foi construída e até colocada em produção, mas a migração das regras e a mudança da forma de trabalho não se estabeleceram, seguindo como um problema futuro a ser abordado, sem que um plano de ação tenha sido sequer traçado, assumindo o custo recorrente de erros e retrabalho, e o gasto com o acoplamento com a equipe da ferramenta de autosserviço. Vale notar que, já na avaliação do ciclo 2, um gerente de equipe nomeava a migração como um desafio de porte modesto,

Tabela 2: As cinco mudanças e o estágio de assimilação em que estagnam (unidade de observação: a mudança).

ID	Mudança	Ciclo	Tipo	Estagna em	Evidência observável
M1	Inventário de funcionalidades	1	Artefato	Aceitação	Implantado e integrado ao gerenciador de regras; uso pontual e fora da rotina, depois despriorizado.
M2	Métrica de adoção (<i>JSONPath</i>)	1	Artefato	Aceitação	Execução automatizada semanal da análise das folhas de regras; a equipe deixou de acompanhar a métrica.
M3	Nova API de configuração de produtos	2	Artefato	Adaptação	Construída e disponibilizada em produção, mas não empregada: a equipe não migrou as regras existentes e segue no paradigma anterior.
M4	Casos de uso problemáticos na ferramenta de experimentação	3	Artefato	Adaptação	Prova de conceito disponível em homologação; não empregada pela equipe na rotina.
M5	Mudança no processo de desenvolvimento, com rastreo entre funcionalidades e suítes de teste	3	Processo	Adoção	Proposta deliberada e avaliada, com a importância reconhecida pela gestão; aval para mudar a forma de trabalho não concedido e sem trabalho alocado.

“pegar as regras e passar esses formatos, tem um trabalhinho aí” (Q2–T4–C1, ciclo 2), o que sugere que o trabalho que falta não era desconhecido de quem o formulava. Caso essa leitura se sustente, o que não se estabeleceu talvez não tenha sido o entendimento do que fazer, e sim a decisão de fazê-lo, possibilidade que a fala indica, mas não demonstra. Já com **D2**, levanta-se a questão da negligência de valor[22], onde a equipe não tem senso de urgência para a gestão dos incidentes, e um pagamento possível que seria o mapeamento do ciclo 3 sequer foi priorizado.

Haja vista a capacidade finita da equipe, a capacidade disponível é consumida na construção e no polimento da ferramenta e em outras prioridades da organização, e não no que de fato pagaria as dívidas, que seria empreender a mudança da forma de trabalho. Dessa forma, nem **D1** nem **D2** são pagas, e ambas persistem pagando juros, pois o gargalo não é o pagamento de uma dívida que esgota o da outra, mas o fato de a capacidade não se converter em mudança de processo. Ao mesmo tempo, isso converge com os achados sobre os impactos de migrações [6, 14], em que a própria mudança da forma de trabalho implica uma migração enquanto se trabalha, o que pode ser disruptivo e distrativo, e ajuda a explicar por que essa mudança é sistematicamente adiada.

Além disso, esse adiamento conversa com as causas de dívidas de processo já mapeadas, sendo uma delas explícita no caso, a despriorização organizacional[22], que opera em diferentes camadas, seja na configuração manual, seja no tratamento reativo de incidentes. Soma-se a isso o aspecto deliberado, onde a decisão de adiar o pagamento de uma dívida, mesmo que racional, não desqualifica a dívida como tal. O mesmo não ocorre com a despriorização da métrica de adoção do ciclo 1, onde a equipe descontinuou um instrumento em que não se vê valor hoje e que também não tem custo recorrente, o que se interpreta como adequação de escopo, e não como dívida, ao menos até que se observe um custo recorrente que a equipe tenha decidido aceitar, o que não é o caso. Ou seja, trata-se da estagnação de uma mudança que não está atrelada a uma dívida de processo em si.

Caso se sustente além deste caso, o mecanismo afeta a gestão da melhoria e os modelos de maturidade em LPS, pois aferir maturidade pela mera existência de instrumentos torna-se enganoso: uma organização pode ter ferramentas em produção enquanto a dívida de processo subjacente persiste. A implicação prática é a de blindar capacidade para o pagamento da dívida, ou seja, reservar esforço

explicitamente para a mudança de processo, e não apenas para a construção, sob pena de o caráter residual do processo perpetuar o adiamento. Como a decisão de adotar uma inovação é, em boa medida, tomada no nível gerencial[14], essa blindagem constitui uma alavanca de gestão.

7.2 Estagnação Multi-Estágio na Assimilação em LPS Extrativa

Para essa análise, emprega-se o modelo de Cooper & Zmud [10], definido na Seção 2.3, como lente analítica, muito semelhante ao que foi feito por Wang et al.[36] para a assimilação de práticas ágeis. Esses estágios são usados para caracterizar os estágios de assimilação das mudanças implementadas e descrever o que se entende por estagnação.

Sob essa lente, define-se operacionalmente a estagnação em um estágio **X** por três condições: a inovação atingiu **X**, havendo evidência observável de que as atividades desse estágio ocorreram (por exemplo, uma API construída e disponibilizada em produção atinge a Adaptação); não progrediu para **X+1** (por exemplo, a equipe não passou a empregá-la no trabalho, mantendo o paradigma anterior); e, na janela observada, não há sinal ativo de movimento nessa direção (trabalho alocado, planejamento, recursos comprometidos ou decisão registrada). Em resumo, o estágio de estagnação é o último estágio atingido com evidência observável de que suas atividades ocorreram, ainda que o produto que o encerra não tenha se concretizado.

É o caso da proposta de mudança do processo de desenvolvimento, deliberada e avaliada sem que o aval fosse concedido, o que a situa na Adoção. A estagnação não é, em si, uma falha: é um estado em um momento específico, que pode ser superado depois. Tampouco é uma rejeição da inovação, e sim um possível adiamento de sua assimilação plena, ainda que permaneça aberta a possibilidade de a inovação vir a ser rejeitada ou de a estagnação se prolongar indefinidamente. Registrar o ponto mais avançado alcançado torna a codificação robusta a percursos não lineares, seja quando as atividades de um estágio se iniciam sem que ele se conclua, seja quando um estágio se concretiza e o uso é posteriormente descontinuado, em linha com o que Wang et al.[36] observam sobre a assimilação de práticas.

Para fins de análise, os ciclos são fragmentados em mudanças distintas, sendo uma mudança um artefato ou processo que poderia

ser assimilado independentemente. Desse modo, cada mudança é tipada como artefato ou processo, e o estágio onde estagnou é identificado com base nos critérios acima. Em suma, a Tabela 2 relaciona, para cada mudança, a progressão técnica alcançada e o estágio em que a assimilação estagna, revelando um padrão multi-estágio: as mudanças param em estágios distintos do modelo e nenhuma alcança a Rotinização. As mudanças de artefato avançam até a disponibilização (Adaptação) ou, quando chegam a ser de fato empregadas, até a Aceitação; já a mudança do processo de desenvolvimento estagna mais cedo, na Adoção. Há, assim, uma relação entre o tipo da mudança e o ponto de parada.

Toda mudança, inclusive as de artefato, requer em algum ponto de sua trajetória um passo organizacional, ou seja, o momento em que a forma de trabalho tem de ser revista para incorporá-la, conforme a Seção 2.3. O que varia com o tipo é a posição desse passo: nas mudanças de artefato ele vem depois da entrega técnica, na passagem da Adaptação para a Aceitação; já na mudança do processo de desenvolvimento ele vem antes da construção, na Adoção. É no grupo focal do ciclo 3 que essa mudança é deliberada, onde a fala de um desenvolvedor pode ser lida como esse passo sendo pesado como escolha: “sempre é uma escolha [...] vai ter um trabalho inicial para depois facilitar ou vai trabalhar demais?” (Q3-T1-C2, ciclo 3).

Ou seja, a fala situa onde a mudança está, sem que daí se conclua por que ela para ali. Sob essa leitura, o padrão da Tabela 2 tem um invariante, sendo ele o de que nenhuma das cinco mudanças cruza esse passo. Por isso, o estágio de assimilação não se lê a partir do estado técnico final, pois mesmo em produção um artefato pode não estar sendo empregado no trabalho. Reconhecer que um artefato está pronto e poderia seguir é, contudo, um julgamento do pesquisador em papel de arquiteto.

As mudanças observadas convergem ainda com temas já documentados na literatura: a criação de soluções caseiras[4, 5], a adequação ao contexto já dominado pela equipe e a integração com os processos já estabelecidos, de modo consistente com o que se observa em pequenas empresas[35]. O fato de a migração não ter sido empreendida é, por sua vez, consistente com o que Berger[6] observa sobre os impactos de migrações.

7.3 Discussão Integrativa

Sobre os relacionamentos entre os achados, C1 oferece uma explicação candidata para o padrão que C2 documenta, enquanto C2 oferece a base empírica em granularidade de estágio sobre a qual C1 e outras hipóteses podem ser testadas.

Mas ao mesmo tempo, C1 pode existir sem C2, pois a dívida pode acumular sem que se mapeiem os estágios de assimilação, como por exemplo, se a equipe simplesmente faz a infusão da nova API na rotina, sem revisar o processo de configuração, então a dívida de processo se perpetua sem que se observasse o padrão de estagnação. E C2 pode existir sem C1, pois o padrão pode ser observado e explicado por mecanismos alternativos. Como por exemplo a priorização gerencial[14] ou a fricção de migração[6].

Dadas as particularidades da LPS extrativa, em específico a baixa maturidade e a origem bottom-up, o processo nunca é tratado como principal, mas sim como residual, sendo construído em torno dos artefatos, sendo os artefatos o foco de atenção e investimento. A

impressão digital desse sucateamento é o que C2 documenta, e convém dizer de forma explícita onde os dois achados se tocam: o passo organizacional que C2 localiza na trajetória de cada mudança é a mesma mudança de processo que C1 afirma não ser bancada. Ou seja, C2 apenas registra que nenhuma das cinco o cruza, e C1 oferece uma explicação candidata para isso, que poderia diferir das outras hipóteses candidatas[4, 6, 14, 25] por levar em consideração o tipo de mudança.

A combinação de C1 e C2 pode também auxiliar na detecção de dívidas de processo, onde a estagnação em um estágio específico pode ser um sinal de que a dívida de processo está presente, e o estágio onde estagnou pode indicar o lado do processo que está sendo sucateado. Por fim, o acoplamento entre capacidade e status residual é uma característica da maturidade extrativa, e em uma LPS proativa/madura, com processo bancado de antemão, o gradiente de estagnação por tipo de mudança deve enfraquecer ou inverter, o que pode ser um teste comparativo para trabalhos futuros e a fronteira de validade externa dessa leitura conjunta.

8 Ameaças à Validade

Há um debate em torno de como avaliar a validade de pesquisa-ação, Herr & Anderson [16] sugerem que não se deve utilizar os mesmos critérios de validade de estudos positivistas ou naturalistas, para não marginalizar ou induzir ao erro os pesquisadores.

Nesta seção, dada a natureza do estudo, adotamos as quatro categorias vindas de Runeson & Höst [31] para estudos de caso: validade de construto, validade interna, validade externa e confiabilidade. Dentro de cada categoria, ancoramos critérios próprios de pesquisa-ação de Herr & Anderson[16], reconhecendo que validade interna no sentido experimental não é o critério apropriado para um estudo intervencionista longitudinal.

Validade de construto. A ameaça mais relevante decorre do papel duplo do pesquisador, que atuou como membro da equipe e principal desenvolvedor da equipe por cerca de 5 anos, sendo ao mesmo tempo fonte de profundidade dos dados e vulnerabilidade. Pois o mesmo julgamento que operacionaliza a *dívida de processo* como construto a vivenciou como prática, de modo que o critério para decidir o que conta como evidência pode coincidir com o de quem produziu os artefatos analisados. Para mitigar essa ameaça, adotamos uma abordagem reflexiva e de triangulação, utilizando questionários, grupos focais e observação participante.

Dentre os instrumentos de coleta, o grupo focal apresenta a ameaça de vieses sociais, como a moderação de críticas diante de colegas, embora a análise temática reflexiva tenha tematizado dissonâncias, e contribui para a validade democrática, pois contempla múltiplas perspectivas e interesses dos diferentes perfis de participantes. Esses perfis foram mantidos anônimos pois alguns dos participantes optaram por não se identificar e a organização impôs restrições de confidencialidade.

Pela mesma assimetria de papel, optou-se conscientemente por não conduzir validação pelos participantes (*member checking*): submeter as interpretações à ratificação de colegas, tendo o pesquisador atuado como arquiteto principal, reproduziria o viés de desejabilidade do grupo focal em vez de controlá-lo. Em seu lugar, a checagem se dá de forma contínua, pela observação participante prolongada e pela triangulação entre os instrumentos já descrita.

Validade interna. Do ponto de vista do estudo de caso, a validade interna no sentido experimental não se aplica, pois o estudo é descritivo e de melhoria, e não explanatório. A hipótese candidata causal C1, embora tenha a forma de mecanismo, é uma hipótese gerada por generalização analítica, cuja plausibilidade é discutida na seção 7, e o teste dessa hipótese fica como trabalho futuro, sob validade externa. A sobreposição entre ciclos, que seria um confundidor em um experimento ou uma variável de ruído, é aqui o próprio fenômeno da dívida de processo.

Sobre a validade de processo[16], que é o análogo de pesquisa-ação da validade interna, para sustentar as interpretações, foram utilizadas diferentes fontes de evidência, como a triangulação intra/inter-ciclo e a manutenção de hipóteses candidatas alternativas para a discussão. Sobre o aprendizado contínuo, cada ciclo sustenta a discussão do ciclo seguinte, tanto em nível da prática quanto da pesquisa, como a mudança dos instrumentos de coleta, que foram refinados. Ademais, até a não adoção das intervenções é convertida em aprendizado.

Validade externa. Os resultados podem ser transferidos analiticamente para outros resultados de LPS extrativa, em específico, os de baixa maturidade, dado o contexto detalhado apresentado na seção 5. Os achados são regidos pela janela de observação dos três ciclos e a extensão temporal de comportamentos de longo prazo não é coberta, a assimilação de mudanças pode vir a se resolver fora da janela. Um dos fenômenos observados, o sucateamento, apresenta uma fronteira de validade externa dependente da maturidade da LPS, que deve ser testada em estudos comparativos futuros.

Confiabilidade. Codificação por um único pesquisador é uma ameaça à confiabilidade, mitigada pelo depósito do *codebook* como forma de auditoria. Além disso, os números reportados são descritivos e não inferenciais, adequação julgada por triangulação e poder informacional, não por *n*. Acerca dos critérios de validade dialógica, a qualidade do estudo foi submetida ao diálogo crítico por meio de uma banca, além da revisão por pares e das discussões entre os autores durante a pesquisa e interpretação dos dados.

Validade de resultado. Esse critério tem uma lente diagnóstica, ao olhar para o resultado da ação, e não corretiva. Várias intervenções estagnaram ou foram despriorizadas, evidência de C1: a não-adoção documentada é a assinatura empírica do fenômeno estudado, e não uma falha de validade. Os problemas dos ciclos foram abordados e o resultado os refinou a um nível mais rico, além de propor soluções.

Risco transversal: autoplágio. O presente trabalho, olha para os três ciclos de pesquisa, sendo que os dois últimos estão, um aceito para publicação[30] e outro em revisão em outros venues. Para mitigar o risco de autoplágio, as contribuições do presente trabalho são apenas as observáveis transversalmente, e a redação foi feita do zero, sem enxertos dos outros trabalhos, com declaração explícita do que difere.

9 Conclusão e Trabalhos Futuros

Neste estudo, foi investigada uma plataforma de *microfrontends* que opera como uma LPS extrativa de baixa maturidade, utilizando uma abordagem de pesquisa longitudinal, estudo de caso e pesquisa-ação. A análise sustenta duas contribuições, ambas derivadas de

um único caso e por isso propostas como construtos candidatos: (C1) a identificação de uma dívida de processo em LPS, proposta como um mecanismo causal candidato que explica a estagnação observada; e (C2) a caracterização de um padrão de estagnação multi-estágio tipado, onde artefatos não passam de *Adaptation* e *Acceptance* enquanto a mudança do processo de desenvolvimento fica retida em *Adoption*.

Então, tem-se que, se a dívida de processo (C1) for válida, o ponto de alavanca não está em demonstrar o valor dos artefatos, mas em blindar capacidade para a conversão de processo, alocando esforço contra o ponto de parada típico de cada tipo de mudança. Além disso, a análise conjunta dos estágios (C2) fornece um mapa de intervenção que orienta onde concentrar esforços para facilitar a transição entre os estágios de aceitação e adoção.

Como trabalhos futuros, propõe-se: (i) abordar as diferentes hipóteses causais rivais, como a restrição de capacidade (C1), a priorização gerencial[14] e fricção de migração[6], para explicar os pontos de parada em cada estágio; (ii) abordar o quadro de Cooper & Zmud [10] como uma ferramenta comparativa para analisar forças específicas de cada estágio na literatura de adoção em ES/SPI; e (iii) promover a pesquisa-ação longitudinal como uma abordagem metodológica para estudar a evolução de LPS em tempo real, permitindo uma compreensão mais profunda das dinâmicas de gestão de variabilidade, e evoluções da LPS.

Além dos caminhos de pesquisa propostos, reconhece-se que são necessários testes empíricos adicionais para validar e consolidar os construtos apresentados. Seja pela replicação em outras LPS extrativas ou pela replicação da categorização tipada dos artefatos, processos e intervenções em outras LPS de maturidades diferentes, a fim de testar a fronteira do sucateamento abordado aqui. Caso padrões análogos não emergam nesses estudos, os construtos devem ser refinados ou substituídos, reforçando a necessidade de uma abordagem científica rigorosa e contínua para o avanço do conhecimento na área.

Disponibilidade de Artefatos

Os instrumentos de coleta, os roteiros dos grupos focais e o *codebook* da análise temática compõem o kit de replicação deste estudo, depositado publicamente no Zenodo: doi:10.5281/zenodo.20948867. As transcrições dos grupos focais e os artefatos internos da plataforma não são disponibilizados, haja vista as restrições de confidencialidade impostas pela organização, conforme registrado na Seção 8.

Referências

- [1] Sven Apel, Don Batorty, Christian Kästner, and Gunter Saake. 2013. *Feature-Oriented Software Product Lines: Concepts and Implementation* (1 ed.). Springer.
- [2] Sven Apel, Sergiy Kolesnikov, Norbert Siegmund, Christian Kästner, and Brady Garvin. 2013. Exploring feature interactions in the wild: the new feature-interaction challenge. In *Proceedings of the 5th International Workshop on Feature-Oriented Software Development* (Indianapolis, Indiana, USA) (FOSD '13). Association for Computing Machinery, New York, NY, USA, 1–8. doi:10.1145/2528265.2528267
- [3] Maider Azanza, Leticia Montalvillo, and Oscar Diaz. 2021. 20 years of industrial experience at SPLC: a systematic mapping study. In *Proceedings of the 25th ACM International Systems and Software Product Line Conference - Volume A* (Leicester, United Kingdom) (SPLC '21). Association for Computing Machinery, New York, NY, USA, 172–183. doi:10.1145/3461001.3473059
- [4] Rabih Bashroush, Muhammad Garba, Rick Rabiser, Iris Groher, and Goetz Botterweck. 2017. CASE Tool Support for Variability Management in Software Product Lines. *ACM Comput. Surv.* 50, 1, Article 14 (March 2017), 45 pages. doi:10.1145/3034827

- [5] Martin Becker, Rick Rabiser, and Goetz Botterweck. 2024. Not Quite There Yet: Remaining Challenges in Systems and Software Product Line Engineering as Perceived by Industry Practitioners. In *Proceedings of the 28th ACM International Systems and Software Product Line Conference (Dommeldange, Luxembourg) (SPLC '24)*. Association for Computing Machinery, New York, NY, USA, 179–190. doi:10.1145/3646548.3672587
- [6] Thorsten Berger, Jan-Philipp Steghöfer, Tewfik Ziadi, Jacques Robin, and Jabier Martinez. 2020. The state of adoption and the challenges of systematic variability management in industry. *Empirical Software Engineering* 25, 3 (2020), 1755–1797.
- [7] Virginia Braun and Victoria Clarke. 2019. Reflecting on reflexive thematic analysis. *Qualitative Research in Sport, Exercise and Health* 11, 4 (2019), 589–597. doi:10.1080/2159676X.2019.1628806
- [8] Ana Eva Chacón-Luna, Antonio Manuel Gutiérrez, José A Galindo, and David Benavides. 2020. Empirical software product line engineering: a systematic literature review. *Information and Software Technology* 128 (2020), 106389.
- [9] Ruzanna Chitchyan, Iris Groher, and Joost Noppen. 2017. Uncovering sustainability concerns in software product lines. *Journal of Software: Evolution and Process* 29, 2 (2017), e1853.
- [10] Randolph B Cooper and Robert W Zmud. 1990. Information technology implementation research: a technological diffusion approach. *Management science* 36, 2 (1990), 123–139.
- [11] Paulo Sergio Medeiros dos Santos and Guilherme Horta Travassos. 2009. Action research use in software engineering: An initial survey. In *2009 3rd international Symposium on empirical software Engineering and measurement*. IEEE, 414–417.
- [12] Paulo Sergio Medeiros Dos Santos and Guilherme Horta Travassos. 2011. Action research can swing the balance in experimental software engineering. In *Advances in computers*. Vol. 83. Elsevier, 205–276.
- [13] Thomas Fogdal, Helene Scherrebeck, Juha Kuusela, Martin Becker, and Bo Zhang. 2016. Ten years of product line engineering at Danfoss: lessons learned and way ahead. In *Proceedings of the 20th International Systems and Software Product Line Conference (Beijing, China) (SPLC '16)*. Association for Computing Machinery, New York, NY, USA, 252–261. doi:10.1145/2934466.2934491
- [14] G.C. Green and A.R. Hevner. 2000. The successful diffusion of innovations: guidance for software development organizations. *IEEE Software* 17, 6 (2000), 96–103. doi:10.1109/52.895175
- [15] Geir K. Hanssen. 2012. A longitudinal case study of an emerging software ecosystem: Implications for practice and theory. *Journal of Systems and Software* 85, 7 (2012), 1455–1466. doi:10.1016/j.jss.2011.04.020
- [16] K. Herr and G.L. Anderson. 2014. *The Action Research Dissertation: A Guide for Students and Faculty*. SAGE Publications. <https://books.google.com.br/books?id=mlQXBAAQBAJ>
- [17] Maira Marques, Jocelyn Simmonds, Pedro O. Rossel, and Maria Cecilia Bastarrica. 2019. Software product line evolution: A systematic literature review. *Information and Software Technology* 105 (2019), 190–208. doi:10.1016/j.infsof.2018.08.014
- [18] Jabier Martinez, Wesley K. G. Assunção, and Tewfik Ziadi. 2017. ESPLA: A Catalog of Extractive SPL Adoption Case Studies. In *Proceedings of the 21st International Systems and Software Product Line Conference - Volume B (Sevilla, Spain) (SPLC '17)*. Association for Computing Machinery, New York, NY, USA, 38–41. doi:10.1145/3109729.3109748
- [19] Jabier Martinez, Tewfik Ziadi, Tegawendé F. Bissyandé, Jacques Klein, and Yves Le Traon. 2015. Bottom-up adoption of software product lines: a generic and extensible approach. In *Proceedings of the 19th International Conference on Software Product Line (Nashville, Tennessee) (SPLC '15)*. Association for Computing Machinery, New York, NY, USA, 101–110. doi:10.1145/2791060.2791086
- [20] Jabier Martinez, Tewfik Ziadi, Tegawendé F. Bissyandé, Jacques Klein, and Yves Le Traon. 2017. Bottom-Up Technologies for Reuse: Automated Extractive Adoption of Software Product Lines. In *2017 IEEE/ACM 39th International Conference on Software Engineering Companion (ICSE-C)*. 67–70. doi:10.1109/ICSE-C.2017.15
- [21] Jabier Martinez, Tewfik Ziadi, Mike Papadakis, Tegawendé F Bissyandé, Jacques Klein, and Yves Le Traon. 2018. Feature location benchmark for extractive software product line adoption research using realistic and synthetic eclipse variants. *Information and Software Technology* 104 (2018), 46–59.
- [22] Antonio Martini, Terese Besker, and Jan Bosch. 2020. Process Debt: a First Exploration. In *2020 27th Asia-Pacific Software Engineering Conference (APSEC)*. 316–325. doi:10.1109/APSEC51365.2020.00040
- [23] Antonio Martini, Viktoria Stray, and Nils Brede Moe. 2019. Technical-, Social- and Process Debt in Large-Scale Agile: An Exploratory Case-Study. In *Agile Processes in Software Engineering and Extreme Programming - Workshops*, Rashina Hoda (Ed.). Springer International Publishing, Cham, 112–119.
- [24] Judy McKay and Peter Marshall. 2001. The dual imperatives of action research. *Information technology & people* 14, 1 (2001), 46–59.
- [25] Jorge Melegati, Nicolas Nascimento, Rafael Chanin, Afonso Sales, and Igor Wiese. 2024. Exploring potential implications of intelligent tools for human aspects of software engineering. In *Proceedings of the 2024 IEEE/ACM 17th International Conference on Cooperative and Human Aspects of Software Engineering (Lisbon, Portugal) (CHASE '24)*. Association for Computing Machinery, New York, NY, USA, 121–132. doi:10.1145/3641822.3641877
- [26] Andreas Metzger and Klaus Pohl. 2014. Software product line engineering and variability management: achievements and challenges. In *Future of Software Engineering Proceedings (Hyderabad, India) (FOSE 2014)*. Association for Computing Machinery, New York, NY, USA, 70–84. doi:10.1145/2593882.2593888
- [27] Rodrigo André Ferreira Moreira, Wesley KG Assunção, Jabier Martinez, and Eduardo Figueiredo. 2022. Open-source software product line extraction processes: the ArgoUML-SPL and Phaser cases. *Empirical Software Engineering* 27, 4 (2022), 85.
- [28] Kai Petersen, Cigdem Gencel, Negin Asghari, Dejan Baca, and Stefanie Betz. 2014. Action research as a model for industry-academia collaboration in the software engineering context. In *Proceedings of the 2014 International Workshop on Long-Term Industrial Collaboration on Software Engineering (Vasteras, Sweden) (WISE '14)*. Association for Computing Machinery, New York, NY, USA, 55–62. doi:10.1145/2647648.2647656
- [29] Klaus Pohl, Günter Böckle, and Frank Van Der Linden. 2005. *Software product line engineering: foundations, principles, and techniques*. Vol. 1. Springer.
- [30] Rhuan Queiroz and André Barros de Sales. 2026. Pre-adoption Expectations of Development Teams on Product Configuration Automation: A Case Study in Software Product Line. In *Proceedings of the International DMS Conference on Visualization and Visual Languages (DMSVIVA2026)*. KSI Research Inc. Artigo selecionado para publicação em edição especial do Journal of Visual Languages and Computing (JVLC); Paper 90.
- [31] Per Runeson and Martin Höst. 2009. Guidelines for conducting and reporting case study research in software engineering. *Empirical Softw. Engg.* 14, 2 (April 2009), 131–164. doi:10.1007/s10664-008-9102-8
- [32] Gerald I. Susman and Roger D. Evered. 1978. An Assessment of the Scientific Merits of Action Research. *Administrative Science Quarterly* 23, 4 (1978), 582–603. doi:10.2307/2392581
- [33] Amrit Tiwana. 2013. *Platform ecosystems: Aligning architecture, governance, and strategy*. Newnes.
- [34] David Tripp. 2005. Pesquisa-ação: uma introdução metodológica. *Educação e Pesquisa* 31, 3 (dez. 2005), 443–466. doi:10.1590/S1517-97022005000300009
- [35] Micheal Tuape, Victoria T. Hasheela-Mufeti, Anna Kayanda, Jari Porras, and Jussi Kasurinen. 2021. Software Engineering in Small Software Companies: Consolidating and Integrating Empirical Literature Into a Process Tool Adoption Framework. *IEEE Access* 9 (2021), 130366–130388. doi:10.1109/ACCESS.2021.3113328
- [36] Xiaofeng Wang, Kieran Conboy, and Minna Pikkariainen. 2012. Assimilation of agile practices in use. *Information Systems Journal* 22, 6 (2012), 435–455.
- [37] Daniele Wolfart, Wesley Klewerton Guez Assunção, and Jabier Martinez. 2021. Variability Debt: Characterization, Causes and Consequences. In *Proceedings of the XX Brazilian Symposium on Software Quality (Virtual Event, Brazil) (SBQS '21)*. Association for Computing Machinery, New York, NY, USA, Article 17, 10 pages. doi:10.1145/3493244.3493250
- [38] Robert K Yin. 2018. *Case study research and applications*. Vol. 6. Sage Thousand Oaks, CA.
- [39] Jesse Yli-Huumo, Andrey Maglyas, and Kari Smolander. 2014. The sources and approaches to management of technical debt: a case study of two product lines in a middle-size finnish software company. In *International Conference on Product-Focused Software Process Improvement*. Springer, 93–107.
- [40] Jesse Yli-Huumo, Andrey Maglyas, and Kari Smolander. 2016. The effects of software process evolution to technical debt—perceptions from three large software projects. In *Managing Software Process Evolution: Traditional, Agile and Beyond—How to Handle Process Change*. Springer, 305–327.